

My Awesome DB — Complete Implementation Guide & Daily Plan

Course: COL7362 (Database Systems) | **Deadline:** 13th April 2026 (first submission) **Partners:** P1 & P2
| **Language:** Rust | **Start Date:** 20th March 2026

Table of Contents

- 1. [Project Overview & Architecture](#)
- 2. [Tech Stack & Prerequisites](#)
- 3. [Key Concepts for Beginners](#)
- 4. [Codebase Structure & Starter Code](#)
- 5. [Component Deep-Dives](#)
- 6. [Daily Implementation Plan \(P1 & P2\)](#)
- 7. [Pseudocode & Implementation Details](#)
- 8. [Testing & Debugging Guide](#)
- 9. [Optimization Strategies](#)
- 10. [Prompts for AI Assistance](#)

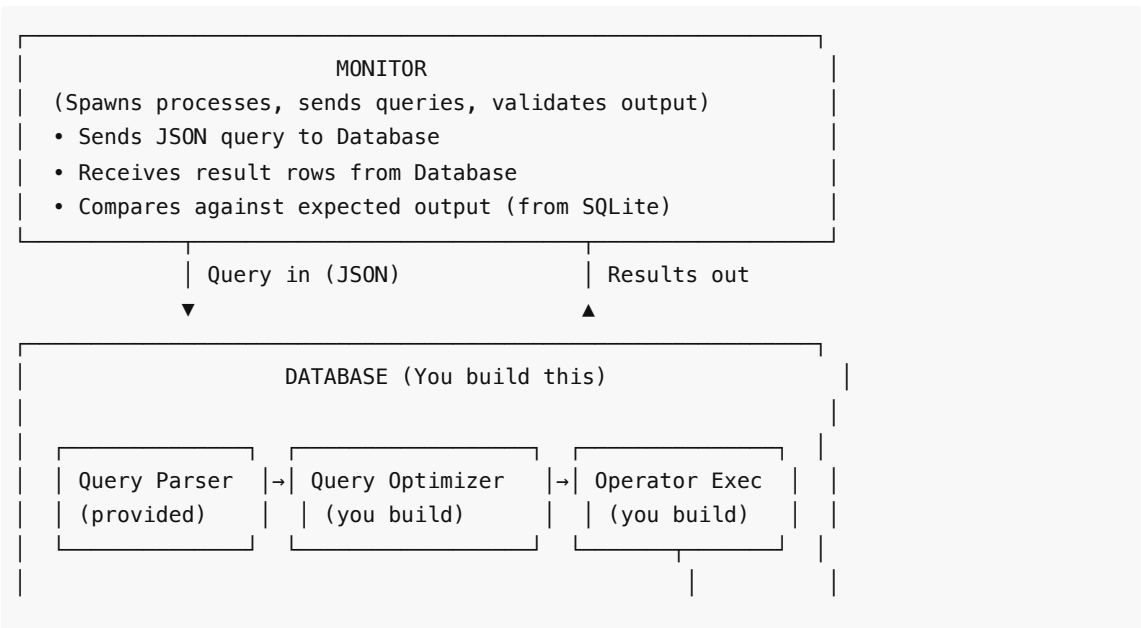
1. Project Overview & Architecture

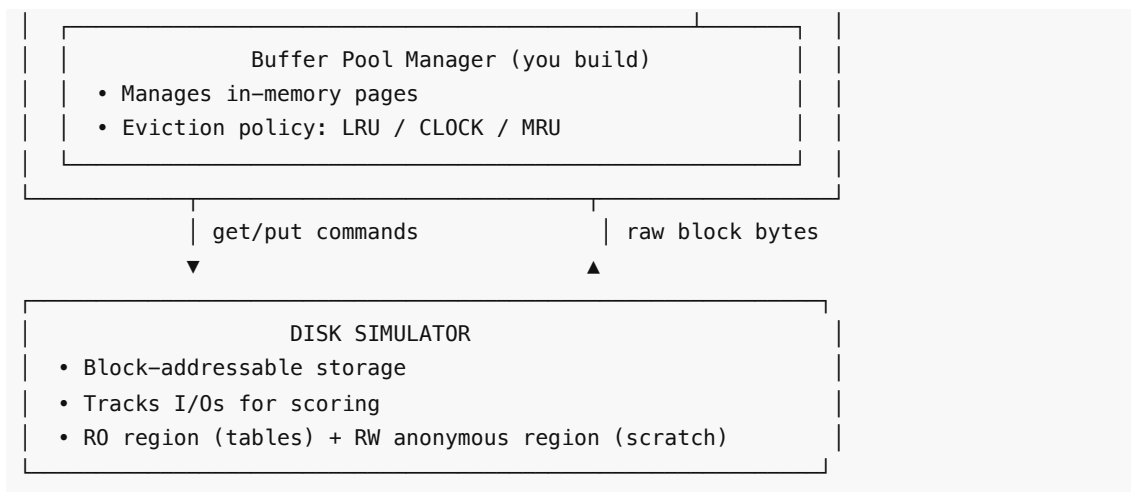
What Are We Building?

A **single-threaded, memory-constrained SQL query engine** called "My Awesome DB" that:

- Receives a **JSON-encoded query** (an AST of Scan/Cross/Filter/Project/Sort operators)
- Reads table data from a **Disk Simulator** (block-addressable storage)
- Executes the query using a **Buffer Pool** for memory management
- Outputs results to a **Monitor** process for validation

Architecture Diagram





Three Processes at Runtime

Process	Role	Communication
Monitor	Sends query, validates output, enforces limits	stdin/stdout with Database
Database	YOUR CODE — parses query, reads disk, outputs rows	stdin/stdout with Monitor; stdin/stdout with Disk
Disk Simulator	Serves blocks, tracks I/O cost	stdin/stdout with Database

Constraints

Constraint	Detail
Memory	≥ 64 MB (query via <code>get_memory_limit</code>)
Threads	Single-threaded only (<code>RLIMIT_NPROC = 1</code>)
File I/O	Forbidden (<code>RLIMIT_FSIZE = 0</code>); use disk simulator's anonymous region
Scoring	Simulated disk I/O time + CPU time

2. Tech Stack & Prerequisites

Rust Basics You Need

Concept	Why It Matters	Quick Resource
Ownership & Borrowing	Core Rust memory model; no GC	Rust Book Ch. 4
<code>struct</code> & <code>enum</code>	Data modeling (rows, columns, operators)	Rust Book Ch. 5-6
<code>Vec<u8></code> , slices	Raw byte manipulation for blocks	Rust Book Ch. 8
Traits	Interfaces for operators (Iterator pattern)	Rust Book Ch. 10

serde / serde_json	Parsing JSON configs and queries	serde.rs
std::io::{BufReader, BufWriter}	Buffered I/O to disk simulator pipes	Rust std docs
byteorder crate	Reading little-endian ints/floats from bytes	docs.rs
Error handling (Result, ?)	Robust I/O handling	Rust Book Ch. 9

Setup Commands

```
# Install Rust
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh

# Install SQLite (for generating expected output)
# macOS:
brew install sqlite3
# Ubuntu:
sudo apt install sqlite3

# Clone starter code
git clone <my-awesome-db-repo-url>
cd my-awesome-db
cargo build -r
```

3. Key Concepts for Beginners

3.1 Block-Addressable Storage

Why blocks? Real disks read/write in fixed-size chunks (e.g., 4096 bytes). Random access is expensive (seek time). Sequential reads are fast.

```
Disk = [ Block 0 | Block 1 | Block 2 | ... | Block N ]
        ← R0 region (tables) → ← RW anonymous (scratch) →
```

- **Block size:** Typically 4096 bytes, queried via `get block-size`
- **Each block** stores multiple rows + a 2-byte footer (row count)
- **Usable space per block** = `block_size - 2` bytes

3.2 Buffer Pool

Why? You can't hold the entire database in memory. The buffer pool is a cache of recently-used disk blocks in RAM.

```
Buffer Pool (fixed size, e.g., 1000 frames)

| Frame 0 | | Frame 1 | | Frame 2 | | ... | | Frame N | ← each frame = one block
└────────┴────────┴────────┴────────┴────────┘
      ⚡ evict/load via disk simulator
```

Eviction policies:

- **LRU** (Least Recently Used): Evict the frame not used for the longest time
- **CLOCK** (Second-chance): Circular scan; give each page a "second chance" bit
- **MRU** (Most Recently Used): Better for sequential scans (avoids polluting cache)

3.3 Query AST (Abstract Syntax Tree)

The monitor sends a JSON query that's a **tree of operators**:

```
Project(columns)      ← outermost: pick & rename columns
├─ Sort(specs)        ← sort by columns
│   └─ Filter(preds) ← WHERE clause conditions
│       └─ Filter(join predicate: a1 = b1)
│           └─ Cross          ← Cartesian product
│               └─ Scan(A)    ← read table A
│                   └─ Scan(B) ← read table B
```

Five operators you must implement:

Operator	SQL Equivalent	Description
Scan(table_id)	FROM table	Read all rows from a table file on disk
Cross(left, right)	FROM A, B	Cartesian product of two sub-queries
Filter(predicates)	WHERE ...	Keep only rows matching all predicates
Project(column_map)	SELECT a AS x	Pick and rename columns
Sort(sort_specs)	ORDER BY ...	Sort rows by specified columns

3.4 Row Encoding (On-Disk Format)

Each block stores rows packed sequentially:

```
[ Row1 | Row2 | Row3 | ... | RowN | padding | cnt ]
cnt = 2 bytes (u16 LE)
```

Each row's columns are concatenated in schema order:

Type	Bytes	Encoding
Int32	4	Signed little-endian
Int64	8	Signed little-endian
Float32	4	IEEE-754 LE
Float64	8	IEEE-754 LE
String	variable	UTF-8 + 0x00 terminator

Example: Schema (id: Int32, name: String), row (0xDEADBEEF, "hi") :

```
EF BE AD DE 68 69 00
^-- id ---^ ^name--^
```

3.5 Join Algorithms (from Lecture)

Since the query has `Cross + Filter(a1 = b1)`, you should **detect this pattern** and use a join algorithm instead of a naive cross product.

Algorithm	Best When	I/O Cost (block transfers)
Nested Loop Join	Small tables	$b_R + n_R \times b_S$
Block Nested Loop	Default fallback	$b_R + \lceil b_R / (B-2) \rceil \times b_S$
Sort-Merge Join	Tables sortable on join key	$\text{sort}(R) + \text{sort}(S) + b_R + b_S$
Grace Hash Join	Large tables, enough memory	$3(b_R + b_S)$

Key insight for optimization: `Cross + Filter(equi-join)` → convert to Hash Join or Sort-Merge Join.

4. Codebase Structure & Starter Code

```
my-awesome-db/
├─ Cargo.toml           # Workspace config
├─ src/
│  ├─ bin/
│  │  ├─ database.rs     # YOUR MAIN ENTRY POINT
│  │  ├─ monitor.rs      # Monitor process (provided)
│  │  ├─ disk.rs         # Disk simulator (provided)
│  │  └─ generator.rs    # Data import tool (provided)
│  └─ lib.rs            # Shared types & utilities
├─ scratch/
│  ├─ datasets/tpch/     # CSV + schema files
│  ├─ compiled_datasets/ # Generated .bin + configs
│  └─ runtimes/          # Monitor configs + expected outputs
└─ target/release/      # Compiled binaries
```

Communication Protocols

Database ↔ Disk Simulator (via stdin/stdout pipes):

Commands you SEND to disk:	Responses you READ from disk:
<code>get block-size\n</code>	→ <code>"4096\n"</code>
<code>get block <id> <count>\n</code>	→ <code><count × block_size raw bytes></code>
<code>get file start-block <fid>\n</code>	→ <code>"<block_id>\n"</code>
<code>get file num-blocks <fid>\n</code>	→ <code>"<num_blocks>\n"</code>
<code>get anon-start-block\n</code>	→ <code>"<block_id>\n"</code>
<code>put block <id> <count>\n<bytes></code>	→ (no response)

Database ↔ Monitor (via stdin/stdout):

READ from monitor:

<JSON query on first line>

WRITE to monitor:

get_memory_limit\n → reads "<MB>\n"
validate\n
col1|col2|col3|\n (per result row)
!\n (end of output)

5. Component Deep-Dives

5.1 Disk I/O Layer

Purpose: Wrap the stdin/stdout communication with the disk simulator into a clean Rust API.

```
// Pseudocode for DiskManager
struct DiskManager {
    reader: BufReader<ChildStdout>, // read from disk process
    writer: BufWriter<ChildStdin>,  // write to disk process
    block_size: usize,
}

impl DiskManager {
    fn get_block_size(&mut self) -> usize { ... }
    fn read_blocks(&mut self, start_id: u64, count: u64) -> Vec<u8> { ... }
    fn write_blocks(&mut self, start_id: u64, data: &[u8]) { ... }
    fn get_file_start_block(&mut self, file_id: &str) -> u64 { ... }
    fn get_file_num_blocks(&mut self, file_id: &str) -> u64 { ... }
    fn get_anon_start_block(&mut self) -> u64 { ... }
}
```

5.2 Buffer Pool Manager

Purpose: Cache disk blocks in memory; evict when full.

```
struct BufferPool {
    frames: Vec<Frame>, // Fixed-size array of frames
    page_table: HashMap<u64, usize>, // block_id → frame_index
    disk: DiskManager,
    eviction_policy: Box<dyn EvictionPolicy>,
}

struct Frame {
    data: Vec<u8>, // block_size bytes
    block_id: Option<u64>,
    dirty: bool, // true if modified (for anon blocks)
    pin_count: u32, // >0 means in use, don't evict
}

impl BufferPool {
    fn fetch_block(&mut self, block_id: u64) -> &[u8] {
        // 1. Check page_table → if found, pin & return
    }
}
```

```

    // 2. Find free frame (or evict via policy)
    // 3. If evicted frame is dirty, write back to disk
    // 4. Read new block from disk into frame
    // 5. Update page_table, pin, return
}
fn unpin(&mut self, block_id: u64) { /* decrement pin_count */ }
fn mark_dirty(&mut self, block_id: u64) { /* set dirty flag */ }
}

```

LRU Implementation (simplest):

```

// Use a LinkedList or VecDeque to track access order
// On access: move to front
// On evict: remove from back (least recently used)
// Only evict frames with pin_count == 0

```

5.3 Row/Tuple Representation

```

#[derive(Clone, Debug)]
enum Value {
    Int32(i32),
    Int64(i64),
    Float32(f32),
    Float64(f64),
    Str(String),
}

struct Row {
    values: Vec<Value>, // one per column, in schema order
}

// Decoding a row from raw bytes:
fn decode_row(bytes: &[u8], schema: &[ColumnSpec]) -> (Row, usize) {
    let mut offset = 0;
    let mut values = Vec::new();
    for col in schema {
        match col.data_type {
            DataType::Int32 => {
                let v =
i32::from_le_bytes(bytes[offset..offset+4].try_into().unwrap());
                values.push(Value::Int32(v));
                offset += 4;
            }
            DataType::Int64 => {
                let v =
i64::from_le_bytes(bytes[offset..offset+8].try_into().unwrap());
                values.push(Value::Int64(v));
                offset += 8;
            }
            DataType::Float32 => {

```

```

        let v =
f32::from_le_bytes(bytes[offset..offset+4].try_into().unwrap());
        values.push(Value::Float32(v));
        offset += 4;
    }
    DataType::Float64 => {
        let v =
f64::from_le_bytes(bytes[offset..offset+8].try_into().unwrap());
        values.push(Value::Float64(v));
        offset += 8;
    }
    DataType::String => {
        let end = bytes[offset..].iter().position(|&b| b == 0).unwrap();
        let s =
String::from_utf8(bytes[offset..offset+end].to_vec()).unwrap();
        values.push(Value::Str(s));
        offset += end + 1; // +1 for null terminator
    }
}
}
(Row { values }, offset)
}

```

5.4 Table Scanner

```

struct TableScanner {
    table: TableSpec,
    file_start_block: u64,
    file_num_blocks: u64,
    current_block: u64, // which block we're reading
    current_row_in_block: u16, // which row within current block
    rows_in_current_block: u16,
}

impl Iterator for TableScanner {
    type Item = Row;
    fn next(&mut self) -> Option<Row> {
        loop {
            if self.current_block >= self.file_start_block + self.file_num_blocks {
                return None; // done
            }
            let block_data = buffer_pool.fetch_block(self.current_block);
            let row_count = u16::from_le_bytes(
                block_data[block_size-2..block_size].try_into().unwrap()
            );
            if self.current_row_in_block < row_count {
                // decode row at current offset
                let row = decode_row_at_index(...);
                self.current_row_in_block += 1;
                return Some(row);
            } else {

```

```

        // move to next block
        self.current_block += 1;
        self.current_row_in_block = 0;
    }
}
}
}

```

5.5 Operator Implementations

Filter (Volcano/iterator model):

```

struct FilterOp {
    child: Box<dyn Operator>,
    predicates: Vec<Predicate>,
}

impl Operator for FilterOp {
    fn next(&mut self) -> Option<Row> {
        while let Some(row) = self.child.next() {
            if self.predicates.iter().all(|p| p.evaluate(&row)) {
                return Some(row);
            }
        }
        None
    }
}

```

Project:

```

struct ProjectOp {
    child: Box<dyn Operator>,
    column_map: Vec<(String, String)>, // (input_name, output_name)
}

impl Operator for ProjectOp {
    fn next(&mut self) -> Option<Row> {
        self.child.next().map(|row| {
            // pick only the columns in column_map and rename
            let new_values = self.column_map.iter()
                .map(|(input, _)| row.get_column(input).clone())
                .collect();
            Row { values: new_values }
        })
    }
}

```

Sort (External Sort — critical for large tables):

```

// Phase 1: Create sorted runs
// - Read B pages worth of rows into memory
// - Sort in-memory
// - Write sorted run to anonymous disk blocks
// - Repeat until all input consumed

// Phase 2: Merge runs
// - Open one buffer page per run (up to B-1 runs)
// - Use min-heap to merge
// - Output merged rows
// If more than B-1 runs, do multi-pass merge

fn external_sort(input: &mut dyn Operator, sort_specs: &[SortSpec]) ->
SortedIterator {
    let mut runs: Vec<Run> = Vec::new();

    // Phase 1: create sorted runs
    loop {
        let mut buffer = Vec::new();
        // Fill buffer with as many rows as fit in memory
        while memory_available() && let Some(row) = input.next() {
            buffer.push(row);
        }
        if buffer.is_empty() { break; }
        buffer.sort_by(|a, b| compare_by_sort_specs(a, b, sort_specs));
        let run = write_run_to_anon_blocks(&buffer);
        runs.push(run);
    }

    // Phase 2: merge runs using min-heap
    merge_runs(runs, sort_specs)
}

```

5.6 Join Implementation (Optimization of Cross + Filter)

When to convert: If the AST has `Filter(a = b) → Cross(L, R)` where `a` from `L` and `b` from `R`, this is an equi-join.

Block Nested Loop Join (simplest correct approach):

```

fn block_nested_loop_join(left: &mut dyn Operator, right: &mut dyn Operator,
    join_col_left: &str, join_col_right: &str) ->
JoinIterator {
    // For each chunk of B-2 blocks from left:
    //   Build hash map: join_key → Vec<Row>
    //   For each row in right:
    //     Probe hash map
    //     Emit matches
}

```

Grace Hash Join (better for large tables):

```
fn grace_hash_join(left: &mut dyn Operator, right: &mut dyn Operator,
                  join_col_left: &str, join_col_right: &str) -> JoinIterator {
    // Phase 1: PARTITION
    // Hash both relations into N partitions on join key
    // Write partitions to anonymous disk blocks

    // Phase 2: BUILD & PROBE
    // For each partition i:
    //   Load left partition i into in-memory hash table
    //   Scan right partition i, probe hash table
    //   Emit matches
}
```

6. Daily Implementation Plan (P1 & P2)

Timeline: March 20 — April 13, 2026 (25 days, ~3.5 weeks)

Phase 1: Foundation (March 20–26) — Week 1

Day 1 (March 20) — Setup & Rust Basics

Partner	Task	Details
P1	Environment setup + Rust basics	Install Rust, clone repo, cargo build -r, read Rust Book Ch. 1-4 (ownership)
P2	Environment setup + understand architecture	Install Rust + SQLite, read assignment PDF thoroughly, draw architecture diagram
Both	Sync meeting (30 min)	Walk through the assignment together, clarify doubts

Commit: feat: initial project setup and cargo build verified

Day 2 (March 21) — Data Import & Exploration

Partner	Task	Details
P1	Import TPCB dataset	Download tpch_scratch.tar.gz, run tar -xf, run generator to create .bin files and configs
P2	Study starter code	Read database.rs, lib.rs, understand Query/TableSpec/ColumnSpec structs, how serde parsing works
Both	Verify setup	Run monitor with a simple Scan query, observe the communication flow

Commit: feat: import TPCB dataset and verify monitor-database pipeline

Day 3 (March 22) — Disk I/O Layer

Partner	Task	Details
P1	Implement DiskManager struct	get_block_size(), get_anon_start_block(), get_file_start_block(), get_file_num_blocks()

P2	Implement <code>read_blocks()</code> and <code>write_blocks()</code>	Handle raw byte I/O, flushing, correct protocol formatting
Both	Test: read a known block and print raw bytes	Verify bytes match expected data from CSV

Commit: feat: disk I/O layer with read/write block support

Day 4 (March 23) — Row Decoding

Partner	Task	Details
P1	Implement <code>Value</code> enum and <code>decode_row()</code>	Handle <code>Int32</code> , <code>Int64</code> , <code>Float32</code> , <code>Float64</code> , <code>String</code> decoding from raw bytes
P2	Implement block parsing	Read <code>row_count</code> from footer, iterate rows in a block, handle padding
Both	Test: decode all rows from block 0 of customer table, print them	Compare with CSV

Commit: feat: row decoding and block parsing for all data types

Day 5 (March 24) — Table Scanner

Partner	Task	Details
P1	Implement <code>TableScanner</code> as iterator	Use <code>DiskManager</code> to read blocks sequentially, return decoded rows
P2	Implement the operator trait	Define trait <code>Operator { fn next() -> Option<Row>; },</code> wrap scanner
Both	Test: full table scan of customer, output all rows	Verify count and data integrity

Commit: feat: table scan operator with iterator pattern

Day 6 (March 25) — Monitor Protocol (Output)

Partner	Task	Details
P1	Implement result output to monitor	<code>validate\n</code> , format each row as <code>`col1</code>
P2	Implement <code>get_memory_limit</code> query to monitor	Parse memory limit response, store for buffer pool sizing
Both	End-to-end test: Scan query → pass monitor validation	First green checkmark!

Commit: feat: monitor output protocol – first successful Scan validation ✅

Day 7 (March 26) — Buffer Pool (Basic)

Partner	Task	Details
---------	------	---------

P1	Implement BufferPool struct with Frames	Fixed-size frame array, page_table HashMap, fetch_block
P2	Implement LRU eviction policy	Track access order, evict unpinned frames, write back dirty frames
Both	Integrate buffer pool into TableScanner	All disk reads go through buffer pool now

Commit: feat: buffer pool manager with LRU eviction

Phase 2: Core Operators (March 27 — April 2) — Week 2

Day 8 (March 27) — Filter Operator

Partner	Task	Details
P1	Implement predicate evaluation	EQ, NE, GT, GTE, LT, LTE for all data types, including Column vs Literal comparison
P2	Implement FilterOp	Wraps child operator, applies all predicates, passes matching rows
Both	Test: Scan + Filter query, validate with monitor	

Commit: feat: filter operator with all comparison operators

Day 9 (March 28) — Project Operator

Partner	Task	Details
P1	Implement ProjectOp	Column selection and renaming from column_name_map
P2	Implement schema tracking	Each operator tracks its output schema (column names + types) for downstream use
Both	Test: Scan + Filter + Project query, validate	

Commit: feat: project operator with column renaming

Day 10 (March 29) — Cross (Naive) + Query Tree Builder

Partner	Task	Details
P1	Implement naive CrossOp	Materialize left child, for each left row iterate right child (reset right each time)
P2	Build the query tree from JSON AST	Recursive function: build_operator(ast_node) -> Box<dyn Operator>
Both	Test: 2-table join query (Cross + Filter), validate with small dataset	

Commit: feat: cross product and recursive query tree builder

Day 11 (March 30) — External Sort (Phase 1)

Partner	Task	Details
P1	Implement sorted run creation	Read rows into memory buffer, sort with sort_by, write to anonymous blocks
P2	Implement row serialization/deserialization for anon blocks	Encode rows to bytes for writing to scratch space, decode when reading back
Both	Test: create sorted runs from a table	

Commit: feat: external sort phase 1 – sorted run creation

Day 12 (March 31) — External Sort (Phase 2)

Partner	Task	Details
P1	Implement k-way merge with min-heap	BinaryHeap with custom comparator, merge from multiple runs
P2	Implement multi-pass merge (if needed)	When runs > B-1, merge in multiple passes
Both	Test: Sort query on full customer table, validate order	

Commit: feat: external sort phase 2 – k-way merge complete

Day 13 (April 1) — Integration & Multi-Query Testing

Partner	Task	Details
P1	Test all operator combinations	Scan, Filter, Project, Sort in various orderings
P2	Write SQL queries, generate expected output with SQLite	Use the trailing pipe trick: SELECT a, b, '' FROM ...
Both	Fix bugs, ensure correct output for 5+ different queries	

Commit: test: validated 5+ queries across all operators

Day 14 (April 2) — Buffer & Cleanup

Partner	Task	Details
Both	Bug-fix day, code cleanup, add comments	Address any failing test cases, clean up error handling

Commit: fix: address edge cases in sort and filter operators

Phase 3: Join Optimization (April 3–8) — Week 3

Day 15 (April 3) — Join Detection

Partner	Task	Details
P1	Implement join pattern detection in query tree builder	Detect <code>Filter(col_A = col_B)</code> on top of <code>Cross</code> where A and B come from different children
P2	Implement <code>JoinOp</code> trait and Block Nested Loop Join	Use B-2 pages for outer, 1 page for inner, 1 for output
Both	Test: equi-join query, compare BNLJ vs naive <code>Cross+Filter</code>	

Commit: feat: join detection and block nested loop join

Day 16 (April 4) — Hash Join (Grace)

Partner	Task	Details
P1	Implement partition phase	Hash both relations into N partitions, write to anonymous blocks
P2	Implement build & probe phase	For each partition pair, build hash table on smaller side, probe with larger
Both	Test: join on TPCB tables (customer $\bowtie$ orders)	

Commit: feat: grace hash join implementation

Day 17 (April 5) — Sort-Merge Join

Partner	Task	Details
P1	Implement sort-merge join	Sort both inputs on join key, merge with two-pointer technique
P2	Handle duplicate keys in merge	When join keys match, handle the case where multiple rows share the same key
Both	Test and compare performance with hash join	

Commit: feat: sort-merge join implementation

Day 18 (April 6) — Predicate Pushdown

Partner	Task	Details
P1	Implement predicate pushdown optimization	Move filter predicates as close to Scan as possible in the query tree
P2	Implement statistics-based decisions	Read stats (<code>CardinalityStat</code> , <code>DensityStat</code>) to choose join order (smaller table as outer)

Both	Benchmark: measure I/O count reduction	
-------------	--	--

Commit: feat: predicate pushdown and statistics-based join order

Day 19-20 (April 7–8) — Float Formatting + Edge Cases

Partner	Task	Details
P1	Implement SQLite-compatible float formatting	Match SQLite's float output exactly (no trailing zeros, etc.)
P2	Handle edge cases	Empty tables, single-row tables, NULL-like values, very long strings
Both	Run full TPCB benchmark suite	

Commit: fix: SQLite-compatible float output and edge case handling

Phase 4: Polish & Submit (April 9–13) — Final Week

Day 21-22 (April 9–10) — Performance Tuning

Partner	Task	Details
P1	Optimize buffer pool	Tune frame count based on memory limit, experiment with CLOCK vs LRU
P2	Optimize I/O patterns	Batch sequential reads (multi-block get block), minimize seeks
Both	Profile with cargo build --profile profiling	

Commit: perf: buffer pool and I/O optimization

Day 23 (April 11) — Query Optimizer

Partner	Task	Details
P1	Implement join order optimization	For multi-table joins, try to join smallest tables first
P2	Implement projection pushdown	Project early to reduce row width → more rows per block
Both	Test with complex multi-table queries	

Commit: feat: basic query optimizer with join reordering

Day 24 (April 12) — Full Testing

Partner	Task	Details
---------	------	---------

Both	Comprehensive testing with all TPCCH queries	Ensure Bronze tier (correct output within time limit)
Both	Fix any remaining bugs, clean up code	

Commit: test: full TPCCH validation pass

Day 25 (April 13) — Submission Day 🎉

Partner	Task	Details
Both	Final cargo build -r, run all test cases one more time	Ensure clean build, no warnings
Both	Submit	Upload to submission system

Commit: release: v1.0 – first submission

7. Pseudocode & Implementation Details

7.1 Main Entry Point (database.rs)

```
fn main() {
    // 1. Read JSON query from monitor (stdin line 1)
    let query_json = read_line_from_stdin();
    let query: Query = serde_json::from_str(&query_json).unwrap();

    // 2. Read db_config.json (path from env or args)
    let db_config: DbConfig = load_db_config();

    // 3. Initialize disk manager (connect to disk simulator via fd/pipes)
    let disk = DiskManager::new(/* fd3 read, fd4 write – see starter code */);

    // 4. Query memory limit from monitor
    write_to_monitor("get_memory_limit\n");
    let mem_limit_mb: u64 = read_line_from_monitor().parse().unwrap();

    // 5. Initialize buffer pool
    let block_size = disk.get_block_size();
    let num_frames = (mem_limit_mb * 1024 * 1024) / block_size as u64;
    let buffer_pool = BufferPool::new(num_frames as usize, block_size, disk);

    // 6. Build operator tree from AST
    let mut root_op = build_operator(&query.root, &db_config, &buffer_pool);

    // 7. Execute and output
    write_to_monitor("validate\n");
    while let Some(row) = root_op.next() {
        let line = format_row(&row, &root_op.output_schema());
        write_to_monitor(&line);    // "col1|col2|...|colN|\n"
    }
    write_to_monitor("!\n");
}
```

```
flush_monitor();
}
```

7.2 Predicate Evaluation

```
fn evaluate_predicate(pred: &Predicate, row: &Row, schema: &Schema) -> bool {
    let left_val = row.get_by_name(&pred.column_name, schema);
    let right_val = match &pred.value {
        PredicateValue::Column(name) => row.get_by_name(name, schema),
        PredicateValue::I32(v) => Value::Int32(*v),
        PredicateValue::I64(v) => Value::Int64(*v),
        PredicateValue::F32(v) => Value::Float32(*v),
        PredicateValue::F64(v) => Value::Float64(*v),
        PredicateValue::String(v) => Value::Str(v.clone()),
    };
    match pred.operator {
        Op::EQ => left_val == right_val,
        Op::NE => left_val != right_val,
        Op::GT => left_val > right_val,
        Op::GTE => left_val >= right_val,
        Op::LT => left_val < right_val,
        Op::LTE => left_val <= right_val,
    }
}
```

7.3 Anonymous Block Allocator

```
struct AnonAllocator {
    next_free: u64, // starts at anon_start_block
}

impl AnonAllocator {
    fn allocate(&mut self, num_blocks: u64) -> u64 {
        let start = self.next_free;
        self.next_free += num_blocks;
        start
    }
    // For reuse, maintain a free list:
    fn free(&mut self, start: u64, num_blocks: u64) {
        // add to free list for reuse
    }
}
```

8. Testing & Debugging Guide

Creating Test Queries

```
# 1. Write SQL query
echo "SELECT c_custkey, c_name, ' ' FROM customer WHERE c_custkey < 10;" > query1.sql

# 2. Generate expected output
sqlite3 scratch/compiled_datasets/tpch/sqlite.db < query1.sql >
scratch/runtimes/tpch/expected_1.csv

# 3. Create the JSON query (manually or with a helper)
# For a simple scan + filter:
{
  "root": {
    "Filter": {
      "predicates": [{"column_name": "c_custkey", "operator": "LT", "value": {"I64":
10}}],
      "underlying": {"Scan": {"table_id": "customer"}}
    }
  }
}

# 4. Update monitor_config.json and run
cargo run -r --bin monitor -- --config scratch/runtimes/tpch/monitor_config.json
```

Debugging Tips

1. **Hex dump blocks:** Print first block of each table in hex to verify decoding
2. **Row count check:** Compare your total row count per table vs `wc -l` on CSV
3. **Line-by-line diff:** `diff <(your_output) <(expected_output)`
4. **Single-query testing:** Set `"disabled": true` on all but one query in config
5. **Add logging to stderr:** Monitor only validates stdout; use `eprintln!()` for debug output

9. Optimization Strategies

For I/O (affects simulated disk time):

Strategy	Impact	Difficulty
Sequential reads (multi-block get block)	High — reduces seeks	Easy
Buffer pool (avoid re-reading blocks)	High	Medium
Predicate pushdown (filter early)	High — less data to process	Medium
Join algorithm selection	Very High	Medium
Projection pushdown (narrow rows early)	Medium — more rows/block	Easy

For CPU (affects code execution time):

Strategy	Impact
Avoid cloning rows unnecessarily	Medium

Use <code>&[u8]</code> slices instead of allocating	High
Compare column indices instead of names	Medium
Use <code>BinaryHeap</code> for merge sort	Easy

Priority Order for Bronze Tier:

1. ☒ Correct output (all operators work)
2. ☒ Terminate within time limit
3. Sequential block reads
4. Buffer pool

For Silver+:

5. Join detection (Cross+Filter → Join)
6. Hash join or sort-merge join
7. Predicate pushdown
8. Statistics-based optimization

10. Prompts for AI Assistance

Use these prompts if you get stuck on a specific step:

Disk I/O Layer

"I'm building a Rust database that communicates with a disk simulator via `stdin/stdout` pipes. The protocol is text-based commands like `get block-size\n` that return text, and `get block <id> <count>\n` that returns raw bytes. Write me a `DiskManager` struct with `BufReader/BufWriter` that implements `get_block_size()`, `read_blocks(start_id, count)`, and `write_blocks(start_id, data)`. The `read_blocks` must read exactly `count * block_size` raw bytes after sending the command."

Buffer Pool

"Implement an LRU buffer pool in Rust for a database system. It should have a fixed number of frames (each holding one disk block). Implement `fetch_block(block_id) -> &[u8]` that checks a `HashMap` page table, evicts LRU unpinned frame if needed (writing dirty frames back via `DiskManager`), loads the requested block, and pins it. Also implement `unpin(block_id)` and `mark_dirty(block_id)`."

Row Decoding

"I have raw bytes from a disk block. The block has rows packed from byte 0, with a 2-byte u16 LE row count at the last 2 bytes. Each row has columns in order: `Int32` (4 bytes LE), `Int64` (8 bytes LE), `Float32` (4 bytes IEEE-754 LE), `Float64` (8 bytes LE), `String` (null-terminated UTF-8). Write a Rust function `decode_rows(block_data: &[u8], schema: &[ColumnSpec]) -> Vec<Row>` that decodes all rows."

External Sort

"Implement external merge sort in Rust for a database with limited memory. I have an input iterator of `Row` `s` and access to anonymous disk blocks via `DiskManager`. Phase 1: fill memory buffer with rows,

sort, write sorted run to anonymous blocks. Phase 2: k-way merge using BinaryHeap. I need to handle the case where there are more runs than available buffer pages (multi-pass merge)."

Join Detection

"Given a query AST with Filter(predicates) over Cross(left, right), detect if this is an equi-join. A predicate is an equi-join if it compares a Column from the left child's schema with a Column from the right child's schema using EQ. Return the join columns if detected."

Grace Hash Join

"Implement Grace hash join in Rust. Phase 1 (Partition): hash both relations into N buckets, writing each bucket to anonymous disk blocks. Phase 2 (Build & Probe): for each bucket pair, load the smaller bucket into an in-memory hash table, then scan the larger bucket and probe for matches. Return an iterator of joined rows."

Float Formatting

"I need to format Rust f32 and f64 values to match SQLite's output format. SQLite uses `printf("%.15g", value)` for doubles and `printf("%.7g", value)` for floats. Write a Rust function that replicates this exact behavior."

Quick Reference Card

Disk Commands (Database → Disk Simulator)

```
get block-size\n      → "<size>\n"
get block <ID> <N>\n      → <N * size raw bytes>
get file start-block <FID>\n      → "<block_id>\n"
get file num-blocks <FID>\n      → "<count>\n"
get anon-start-block\n      → "<block_id>\n"
put block <ID> <N>\n<bytes>      → (no response)
```

Monitor Protocol (Database → Monitor)

```
get_memory_limit\n      → "<MB>\n"
validate\n      (start output)
col1|col2|...|colN|\n      (per row)
!\n      (end output)
```

Building & Running

```
cargo build -r                                # build all
cargo run -r --bin monitor -- --config <path> # run with monitor
cargo build --profile profiling                # for Valgrind profiling
```

Remember: The first deadline (April 13) is about **correctness** (Bronze tier). You can re-submit until April 27 to climb the leaderboard. Start with a working solution, then optimize!